Table 1: Summary of GPU Sharding Methods

Method	**Description**	**Pros**	**Cons**
Data Parallelism	Model is replicated on each GPU. Dataset is sharded and each GPU processes a different shard. Gradients are synchronized after each step.	Simple to implement. Efficient for models that fit on a single GPU. Good for large datasets.	Not suitable for models larger than a single GPU's memory. Communication overhead for gradient synchronization can be a bottleneck.
Model Parallelism	Model is split across multiple GPUs. Each GPU is responsible for a part of the model's layers.	Can handle models that are too large for a single GPU.	Can be complex to implement. Can lead to GPU under-utilization if not implemented carefully (the "bubble" effect).
Pipeline Parallelism	A type of model parallelism. The model is split into stages, and each stage is on a different GPU. Data is fed through the pipeline in micro-batches to keep GPUs busy.	Reduces GPU idle time compared to naive model parallelism. Can improve throughput.	Can be complex to balance the stages. Can introduce latency due to the pipeline depth.
Tensor Parallelism	A type of model parallelism. Individual tensors (e.g., weight matrices) are split across multiple GPUs. Each GPU computes a part of the operation.	Allows for very large models to be trained. Can be very efficient for certain operations like large matrix multiplications.	High communication overhead. Typically requires very fast interconnects between GPUs (e.g., NVLink).